密码学核心笔记：高级加密标准 (AES)

密码学学习笔记

2026 年 3 月 30 日

摘要

AES (Advanced Encryption Standard) 即高级加密标准，是目前世界上应用最广泛的对称加密算法。本笔记详细梳理了 AES 的数学基础、算法整体结构以及核心轮函数的运作原理。

1 1. AES 概述

AES 是一种 ** 分组密码 (Block Cipher)**，其前身是 Rijndael 算法。它的设计遵循代数结构，具有高度的安全性和软硬件实现效率。

- **分组长度**：严格固定为 128 bit (16 字节)。
- **密钥长度**：可选 128 bit, 192 bit 或 256 bit。
- **轮数 (Rounds)**：视密钥长度而定，分别为 10 轮、12 轮和 14 轮。

2 2. 核心数学基础：有限域 $GF(2^8)$

AES 的所有字节运算都在伽罗瓦域 $GF(2^8)$ 上进行。一个字节可以表示为一个次数最高为 7 的多项式：

$$b_7x^7 + b_6x^6 + b_5x^5 + b_4x^4 + b_3x^3 + b_2x^2 + b_1x + b_0$$

其中系数 $b_i \in \{0, 1\}$。

运算规则

- **加法与减法**：在 $GF(2^8)$ 中，加法和减法等价于按位异或 (XOR，符号为 $\oplus$)。
- **乘法**：多项式相乘后，必须模一个不可约多项式 $m(x)$。AES 选用的不可约多项式为：

$$m(x) = x^8 + x^4 + x^3 + x + 1 \quad (\text{十六进制记为 } \texttt{0x11B})$$

3 3. 状态矩阵 (State Matrix)

AES 将 128 bit 的输入数据按字节划分为 16 个字节，并排列成一个 4×4 的列优先矩阵，称为 ** 状态矩阵 (State)**。

$$\begin{bmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{bmatrix}$$

后续的所有轮函数操作（除了密钥加法）都是在这个矩阵上进行变换的。

4 4. 轮函数 (Round Function)

除了最后一轮，AES 的标准轮函数包含四个步骤。最后一轮会省略 **MixColumns** 步骤。

4.1 4.1 字节代换 (SubBytes)

这是一个非线性的字节替换过程，通过查 S 盒 (S-box) 独立地替换状态矩阵中的每个字节。设计原理：1. 求字节在 $GF(2^8)$ 中的乘法逆元。2. 进行仿射变换 (Affine Transformation)，破坏代数结构的简单性，抵抗代数攻击。

4.1.1 4.1.1 S 盒的数学构造逻辑

S 盒的设计并非随机生成，而是基于严格的代数结构，其构造过程分为两步：

1. **有限域求逆 (Inverse over $GF(2^8)$)**
   将输入字节 a 映射到其在有限域 $GF(2^8)$ 上的乘法逆元 $b = a^{-1}$。
   - 若 $a \neq 0$，则 $a \cdot b \equiv 1 \pmod{x^8 + x^4 + x^3 + x + 1}$。
   - 若 $a = 0$，则定义其映射结果仍为 0。

   意义：求逆运算具有极高的非线性度，能有效抵抗线性密码分析。

2. **仿射变换 (Affine Transformation)**
   对 b 进行线性变换并异或常数向量。

$$
\begin{bmatrix} s_0 \\ s_1 \\ s_2 \\ s_3 \\ s_4 \\ s_5 \\ s_6 \\ s_7 \end{bmatrix}
=
\underbrace{\begin{bmatrix}
1 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\
1 & 1 & 0 & 0 & 0 & 1 & 1 & 1 \\
1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 \\
1 & 1 & 1 & 1 & 0 & 0 & 0 & 1 \\
1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 \\
0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\
0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 \\
0 & 0 & 0 & 1 & 1 & 1 & 1 & 1
\end{bmatrix}}_{\text{仿射矩阵}}
\begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \\ b_4 \\ b_5 \\ b_6 \\ b_7 \end{bmatrix}
\oplus
\underbrace{\begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \\ 0 \\ 1 \\ 1 \\ 0 \end{bmatrix}}_{\text{0x63}}
$$

注：最后的常数向量对应十六进制 *0x63*。

4.1.2 4.1.2 为什么这样设计?

- **最大化非线性**：仅仅通过求逆得到的变换在代数上过于简单（可以用简单的多项式描述）。仿射变换的引入极大地增加了描述 S 盒所需的布尔代数方程的复杂度，从而抵御代数攻击。
- **消除不动点**：纯求逆运算中 $0 \to 0$。通过异或 `0x63`，使得 $S(00) = 63$，确保了输入为全零时输出不再为零，增强了混淆效果。
- **优异的差分特性**：这种构造保证了最大差分传播概率为 2^{-7}，使得攻击者很难找到高概率的差分路径。

> **小贴士**
>
> 在实际软件实现中，为了提高速度，S 盒通常被预计算并存储为一个 256 字节的查找表（Lookup Table），直接通过输入值作为索引进行快速替换。

4.2 4.2 行移位 (ShiftRows)

在状态矩阵中进行按行的循环左移，提供算法的 ** 扩散性 (Diffusion)**。

- 第 0 行：不移位。
- 第 1 行：循环左移 1 个字节。
- 第 2 行：循环左移 2 个字节。
- 第 3 行：循环左移 3 个字节。

4.3 4.3 列混淆 (MixColumns)

对状态矩阵的每一列进行独立的线性变换。将每一列视为 $GF(2^8)$ 上的四次多项式，乘上一个固定的多项式 $c(x)$ 并模 x^4+1。在矩阵表示中，它等价于左乘一个固定的常数矩阵：

$$\begin{bmatrix} s'_{0,c} \\ s'_{1,c} \\ s'_{2,c} \\ s'_{3,c} \end{bmatrix} = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} s_{0,c} \\ s_{1,c} \\ s_{2,c} \\ s_{3,c} \end{bmatrix}$$

注：矩阵中的数字为十六进制形式，运算均在 $GF(2^8)$ 上进行。

4.4 4.4 轮密钥加 (AddRoundKey)

将当前的状态矩阵与该轮的子密钥 (Round Key) 进行按位异或。

$$S'_{i,j} = S_{i,j} \oplus k_{i,j}$$

5 5. 密钥扩展 (Key Expansion)

AES 算法通过密钥编排方案将初始密钥扩展为一个包含多个字 (Word，1 Word = 4 Bytes) 的数组，为每一轮生成轮密钥。核心操作包括：

- **RotWord**：字的循环移位。
- **SubWord**：对字中的每个字节进行 S 盒替换。
- **Rcon (轮常数)**：消除对称性，其值为 $[x^{i-1}, 00, 00, 00]$。

5.1 5.1 密钥扩展算法

给定一个 N_k 字（$N_k = 4, 6, 8$ 分别对应 128、192、256 位密钥）的初始密钥，扩展生成 $N_r + 1$ 个轮密钥，每个轮密钥由 $N_b = 4$ 字组成（$N_r = 10, 12, 14$）。

扩展后的密钥总字节数为 $N_b \times (N_r + 1) = 4 \times (N_r + 1)$ 字。

密钥扩展核心公式：

$$
w[i] = \begin{cases} \text{Key}[i], & i < N_k \\ w[i-N_k] \oplus \text{SubWord}(\text{RotWord}(w[i-1])) \oplus \text{Rcon}[i/N_k], & i \geqslant N_k \text{ 且} i \bmod N_k = 0 \\ w[i-N_k] \oplus w[i-1], & \text{其他情况} \end{cases} \tag{1}
$$

其中：

- $w[i]$ 表示扩展后的第 i 个字（0-indexed）
- N_k 为初始密钥的字数（4、6 或 8）
- $\text{RotWord}([a_0, a_1, a_2, a_3]) = [a_1, a_2, a_3, a_0]$（循环左移 1 字节）
- SubWord 对字的每个字节执行 S 盒替换

5.2 5.2 轮常数 Rcon

Rcon（Round Constant）用于消除密钥扩展中的对称性，防止轮密钥之间存在简单的代数关系。

Rcon 表格

轮数 i	**Rcon**$[i]$ (十六进制)	x^{i-1} **在** $GF(2^8)$ **中的值**
1	01 00 00 00	$x^0 = 1$
2	02 00 00 00	$x^1 = 2$
3	04 00 00 00	$x^2 = 4$
4	08 00 00 00	$x^3 = 8$
5	10 00 00 00	$x^4 = 16$
6	20 00 00 00	$x^5 = 32$
7	40 00 00 00	$x^6 = 64$
8	80 00 00 00	$x^7 = 128$
9	1B 00 00 00	$x^8 = x^4 + x^3 + x + 1$
10	36 00 00 00	$x^9 = x^5 + x^4 + x^2 + x$

注：Rcon 的计算采用 x^{i-1} 模不可约多项式 $m(x) = x^8 + x^4 + x^3 + x + 1$。

5.3 5.3 128 位密钥扩展示例

假设初始密钥为（十六进制）：

$$\text{Key} = \text{2B 7E 15 16 28 AE D2 A6 AB F7 15 88 09 CF 4F 3C}$$

则前 4 个字即为初始密钥本身：$w[0]$ 到 $w[3]$。
计算 $w[4]$：

1. $i = 4, i \mod N_k = 0$，使用第一种公式
2. $\text{RotWord}(w[3]) = \text{RotWord}(\text{09 CF 4F 3C}) = \text{CF 4F 3C 09}$
3. SubWord(CF 4F 3C 09) 通过 S 盒替换得 8A AE 6D 2B
4. $\text{Rcon}[1] = \text{01 00 00 00}$
5. $w[4] = w[0] \oplus \text{SubWord}(\text{RotWord}(w[3])) \oplus \text{Rcon}[1]$
6. $w[4] = \text{2B 7E 15 16} \oplus \text{8A AE 6D 2B} \oplus \text{01 00 00 00} = \text{A0 FA FE 17}$

依此类推，可生成所有 11 个轮密钥（对应 10 轮加密）。

5.4 5.4 密钥扩展的设计考量

1. **密钥雪崩效应**：初始密钥的 1 bit 变化会导致后续所有轮密钥发生不可预测的改变。
2. **抵抗已知攻击**： Rcon 的引入确保了不同轮次的轮密钥之间不存在简单的线性关系。
3. **实现效率**：扩展算法仅使用基本的异或和 S 盒查表操作，便于硬件和软件实现。
4. **内存优化**：对于 128 位密钥，可在运行时仅保存前 4 字和上一字，边扩展边使用。

5.5 5.5 三种密钥长度的对比

参数对比

密钥长度	密钥字数 N_k	轮数 N_r	轮密钥字数
128 位	4	10	44 字 (176 字节)
192 位	6	12	52 字 (208 字节)
256 位	8	14	60 字 (240 字节)

对于 192 位和 256 位密钥，密钥扩展算法在每 N_k 和 $2N_k$ 字处额外使用 SubWord 变换，进一步增强安全性。

5.6 5.6 从轮密钥恢复主密钥

密钥扩展是一个单向函数，但从某一轮的轮密钥可以逆向推导主密钥（主密钥）。这在密钥管理中有重要应用，例如从内存中提取轮密钥后恢复原始密钥。

> **核心思想**
>
> 由于密钥扩展算法是确定性的且可逆，我们可以通过**逆扩展**操作，从任意一轮的轮密钥向前或向后推导其他轮的轮密钥，最终恢复主密钥。

对于 128 位密钥（10 轮），轮密钥存储结构如下：

$$\underbrace{w[0], w[1], w[2], w[3]}_{\text{主密钥}} \to \underbrace{w[4], \ldots, w[7]}_{\text{Round 1}} \to \underbrace{w[8], \ldots, w[11]}_{\text{Round 2}} \to \cdots \to \underbrace{w[40], \ldots, w[43]}_{\text{Round 10}}$$

5.6.1 逆扩展算法

已知第 r 轮的轮密钥（由 4 个字组成），可以推导出第 $r-1$ 轮的轮密钥：

逆扩展核心公式（128 位密钥）：

$$\begin{aligned}
w[4r] &= w[4r+4] \oplus \mathrm{SubWord}(\mathrm{RotWord}(w[4r+3])) \oplus \mathrm{Rcon}[r] \\
w[4r+1] &= w[4r+5] \oplus w[4r] \\
w[4r+2] &= w[4r+6] \oplus w[4r+1] \\
w[4r+3] &= w[4r+7] \oplus w[4r+2]
\end{aligned}$$

其中 r 是轮数（$1 \leqslant r \leqslant 10$），$w[4r]$ 到 $w[4r+3]$ 表示第 r 轮的轮密钥。

5.6.2 具体示例

假设已知第 10 轮（最后一轮）的轮密钥：

$$\mathrm{Round}_{10} = [w[40], w[41], w[42], w[43]]$$

恢复第 9 轮轮密钥：

1. $w[36] = w[40] \oplus \mathrm{SubWord}(\mathrm{RotWord}(w[39])) \oplus \mathrm{Rcon}[10]$
2. $w[37] = w[41] \oplus w[36]$
3. $w[38] = w[42] \oplus w[37]$
4. $w[39] = w[43] \oplus w[38]$

依此类推，直到恢复出主密钥 $w[0]$ 到 $w[3]$。

5.6.3 密钥恢复的数学原理

- **异或的可逆性**：由于 $a \oplus b = c \Rightarrow a = c \oplus b$，异或运算是其自身的逆运算。
- **SubWord 的可逆性**：S 盒变换是双射（一一对应），存在逆 S 盒（Inverse S-box），可通过查表恢复。
- **RotWord 的可逆性**：循环左移 1 字节的逆操作是循环右移 1 字节。
- **Rcon 的确定性**：轮常数是公开已知的，不增加恢复难度。

5.6.4 安全性 Implications

> **安全警示**
>
> 这一特性意味着：**如果攻击者获取了任意一轮的完整轮密钥，理论上可以恢复出主密钥**。这对密钥存储和内存安全有重要启示：

1. **内存攻击风险**：侧信道攻击、cold boot 攻击等若能提取轮密钥，等同于泄露主密钥。
2. **密钥派生函数 (KDF)**：在实际应用中，应使用 KDF（如 PBKDF2、Scrypt、Argon2）从主密钥派生轮密钥，避免直接使用主密钥进行扩展。
3. **白盒密码**：白盒实现中需对密钥扩展过程进行混淆或使用专用白盒算法，防止密钥恢复。
4. **最佳实践**：敏感场景下，应避免在内存中存储完整的主密钥，仅在需要时临时派生。

5.6.5 扩展到 192 位和 256 位密钥

对于更长密钥，逆扩展算法略有不同：

- **192 位密钥 (12 轮)**：每 6 个字为一组，逆扩展时需考虑 $2N_k$ 位置的额外 SubWord 变换。
- **256 位密钥 (14 轮)**：每 8 个字为一组，每 N_k 和 $2N_k$ 位置都需要 SubWord 变换。

> **统一规律**
>
> **核心原则**：逆扩展与正扩展是**对称**的过程。已知第 r 轮的轮密钥，通过反向迭代（从后往前），每次逆推 N_k 个字，直到恢复主密钥。

6 附加专题：列混合 (MixColumns) 的设计逻辑

核心意义

列混合是 AES 达到 **完全扩散 (Full Diffusion)** 的关键。其设计目标是：在最短的轮数内，让输入状态的每一个比特位都能影响到输出状态的所有比特位。

6.1 1. MDS 矩阵与分支数

列混合本质上是一个线性变换，通过左乘一个常数矩阵 C 实现。

$$S'_{col} = C \cdot S_{col}$$

这个矩阵 C 被选为 **MDS (Maximum Distance Separable)** 矩阵。在编码理论中，MDS 矩阵具有最高的分支数（Branch Number）。

- **分支数定义：** $\mathcal{B} = \min_{x \neq 0}\{\mathrm{wt}(x) + \mathrm{wt}(C \cdot x)\}$，其中 wt 表示非零字节的个数。
- **AES 的表现：** AES 列混合的分支数达到最大值 $k + 1 = 5$。这意味着如果输入列中只有 **1 个字节**发生了改变，输出列的 **4 个字节**必然全部改变。

[Image of AES MixColumns MDS diffusion effect]

6.2 2. 计算效率的平衡

为了在实现复杂度与安全性之间取得平衡，列混合矩阵的系数选用了 `01`，`02`，`03`：

$$\begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix}$$

设计考量：

1. **硬件友好性：** 在 $GF(2^8)$ 中，乘以 `02` 只需一次左移和一次可选的异或（`xtime` 运算），乘以 `03` 只需 (`02` $\oplus$ `01`)。这避免了复杂的乘法器设计。
2. **循环移位特性：** 观察矩阵可以发现，每一行都是前一行循环右移的结果。这种循环对称性极大简化了逻辑电路的布局。

6.3 3. 扩散效果的倍增

列混合（纵向扩散）与 **行移位 (ShiftRows)**（横向扩散）交替进行。这种"纵横交错"的设计确保了：

> 仅需 **两轮**变换，原始明文中的任意 1 bit 变化就会扩散到整个 128 bit 的状态矩阵中。

[Image of AES propagation of a single bit change over rounds]

6.4 4. 解密的对称性挑战

列混合矩阵的逆矩阵系数为 `0E`，`0B`，`0D`，`09`。这些系数比加密系数复杂得多。

- **代价**：导致 AES 解密通常比加密稍慢。
- **权衡**：这种设计将计算压力推向了解密端，而在许多应用场景（如文件加密、流媒体推流）中，加密的性能瓶颈通常比解密更敏感。

7 附加专题：AES 解密算法设计

AES 解密算法是加密算法的逆过程，通过逆向操作恢复明文。解密过程中使用的轮函数是加密轮函数的逆变换。

7.1 1. 解密流程概述

AES 解密（以 128 位密钥为例）包含 10 轮变换：

> **解密流程**
>
> **初始轮**： AddRoundKey（使用第 10 轮轮密钥）
> **主循环 (9 轮)**：InvShiftRows → InvSubBytes → AddRoundKey → InvMixColumns
> **最后一轮**： InvShiftRows → InvSubBytes → AddRoundKey

与加密不同，解密的第一轮和最后一轮的轮函数结构与中间轮相同（均包含 InvMixColumns）。

7.2 2. 逆轮函数组件

2.1 逆字节代换 (InvSubBytes)

InvSubBytes 是 SubBytes 的逆操作，通过逆 S 盒（Inverse S-box）进行字节替换。

逆 S 盒构造

逆 S 盒的构造过程是加密 S 盒的逆操作：

1. 首先进行逆仿射变换（异或 `0x63`）
2. 然后在 $GF(2^8)$ 上求乘法逆元

逆 S 盒的特点：$S^{-1}(S(x)) = x$，即两次变换恢复原值。

2.2 逆行移位 (InvShiftRows)

InvShiftRows 是 ShiftRows 的逆操作，进行循环右移：

- 第 0 行：不移动
- 第 1 行：循环右移 1 个字节
- 第 2 行：循环右移 2 个字节（对于 4×4 矩阵，等价于左移 2 个字节）
- 第 3 行：循环右移 3 个字节

2.3 逆列混淆 (InvMixColumns)

InvMixColumns 是 MixColumns 的逆操作，使用逆矩阵进行变换：

$$\begin{bmatrix} s'_{0,c} \\ s'_{1,c} \\ s'_{2,c} \\ s'_{3,c} \end{bmatrix} = \begin{bmatrix} 0E & 0B & 0D & 09 \\ 09 & 0E & 0B & 0D \\ 0D & 09 & 0E & 0B \\ 0B & 0D & 09 & 0E \end{bmatrix} \begin{bmatrix} s_{0,c} \\ s_{1,c} \\ s_{2,c} \\ s_{3,c} \end{bmatrix}$$

其中系数 `0E`，`0B`，`0D`，`09` 在 $GF(2^8)$ 中定义：

- `0E` $= x^4 + x^3 + x^2 + x$
- `0B` $= x^3 + x + 1$
- `0D` $= x^4 + x^2 + x + 1$
- `09` $= x^3 + 1$

2.4 轮密钥加 (AddRoundKey)

AddRoundKey 是自逆的：$(S \oplus K) \oplus K = S$。因此解密时直接使用相同的轮密钥进行异或即可。

7.3 3. 密钥调度在解密中的应用

解密需要使用与加密相反顺序的轮密钥：

轮密钥顺序

轮数	加密轮密钥	解密轮密钥
初始	K_0	K_{10}
1	K_1	K_9
2	K_2	K_8
$\vdots$	$\vdots$	$\vdots$
9	K_9	K_1
10	K_{10}	K_0

因此，解密时需要预先计算并存储逆序的轮密钥，或者在解密过程中实时逆扩展。

7.4 4. 解密与加密的对比

关键差异

特性	加密	解密
轮数	首轮无 InvMixColumns	首轮无 InvMixColumns
轮顺序	SubBytes → ShiftRows → MixColumns → AddRoundKey	InvShiftRows → InvSubBytes → AddRoundKey → InvMixColumns
轮密钥顺序	$K_0, K_1, \ldots, K_{N_r}$	$K_{N_r}, K_{N_r-1}, \ldots, K_0$
计算复杂度	MixColumns 系数简单 (01,02,03)	InvMixColumns 系数复杂 (09,0B,0D,0E)
执行速度	较快	较慢（约 20-30%）

7.5 5. 解密算法设计的安全性考量

1. **非对称的计算压力**：加密端使用简单系数，解密端使用复杂系数。这是故意的设计选择，因为许多场景中加密需求远大于解密需求。

2. **等密钥长度的变体 (Equivalent Inverse)**：为了使解密与加密结构一致，可以使用「等密钥」技术：将 InvMixColumns 融合到轮密钥中，使解密轮函数形式上与加密相同。

3. **侧信道攻击面**：解密过程中 InvMixColumns 的复杂运算可能泄露更多信息，需要注意实现中的侧信道防护。

5.1 等密钥 (Equivalent Inverse) 技术

等密钥技术允许解密使用与加密相同的轮函数结构，代价是轮密钥需要预处理：

等密钥变换：

$$K'_r = \text{InvMixColumns}(K_r) \quad （对于 r = 1 \text{ 到} N_r - 1）$$

$$K'_0 = K_0, \quad K'_{N_r} = K_{N_r}$$

使用等密钥后，解密流程变为：

$$\text{InvSubBytes} \to \text{InvShiftRows} \to \text{AddRoundKey} \to \text{InvMixColumns}$$

这与加密的结构完全对称，便于实现和优化。

7.6 6. 解密正确性验证

解密正确性的数学基础：

1. **仿射变换可逆**： $S^{-1}(S(x)) = x$（逆 S 盒与 S 盒互为逆函数）
2. **行移位可逆**： $\text{InvShiftRows}(\text{ShiftRows}(S)) = S$
3. **列混淆可逆**： $\text{InvMixColumns}(\text{MixColumns}(S)) = S$
4. **异或自逆**： $(x \oplus k) \oplus k = x$

因此，完整解密过程：

$$\text{Plaintext} = D_K(C) = D_K(E_K(\text{Plaintext})) = \text{Plaintext}$$

---

fill

"Cryptography is about replacing trust with math."